MIT 8103 Advanced Database Systems: Portfolio Summary

Case Study: Digital Banking Platform

Name: Osarenkhoe Chima-Nwogwugwu

Student ID: 301920803

GitHub: grc-cyber-girl/Chima-Nwogwugwu_Osarenkhoe_301920803

Case study

The five portfolios applied database concepts to a Digital Banking Platform supporting customer and account management, authentication, transfers, beneficiaries and transaction history. PostgreSQL was used for structured banking records, while MongoDB was explored for transaction event documents.

Portfolio 1: Database Design and Modelling

I designed the PostgreSQL database `digital_banking_portfolio` with five main tables: `customer`, `account`, `app_user`, `beneficiary` and `bank_transaction`. I identified the platform's users, operations and business rules, including unique customer and account details, one account per customer, KYC before activation and sufficient funds before a transfer.

Primary keys, foreign keys, unique constraints and check constraints helped protect data integrity, including preventing duplicate account numbers and negative balances. I also identified rules—such as customer age and transfer-fund checks—that require application or transaction-level logic.

Portfolio 2: Query Processing and Optimisation

I wrote queries to identify pending transactions, accounts without transactions and transaction counts by customer. I then tested a date-range query against 100,000 synthetic transaction records before and after adding an index on `transaction_date`.

The median execution time fell from 6.942 ms to 0.047 ms. The plan changed from a Sequential Scan to an Index Only Scan, while shared buffer hits on the third run fell from 896 to 3. This showed a substantial improvement for this query and test dataset, not a guarantee for all workloads. Indexes also use storage and can add write overhead.

Portfolio 3: Transactions and Concurrency

I used COMMIT and ROLLBACK to examine transaction behaviour. A customer login-date update was committed. In a separate test, a balance update violated a constraint; rolling back the transaction undid the earlier balance reduction within it.

I also tested concurrent withdrawals using two PostgreSQL sessions at READ COMMITTED. Both attempted to withdraw ₦30,000 from an account holding ₦50,000. Session A updated and committed first. Session B waited for the row lock, then updated zero rows because the remaining balance was insufficient. The final balance was ₦20,000. This demonstrated conditional updates and row locking for withdrawals, not a complete transfer between accounts.

Portfolio 4: NoSQL and Advanced Data Models

I installed MongoDB Community Server in an Ubuntu 24.04 GitHub Codespace and confirmed the server was running. In the `digital_banking_events` database, I created and queried three event documents representing completed, pending and failed transfers. I used queries to count documents, find events for a sender and retrieve successful events.

I initially found the event structure and queries difficult to understand. Reviewing examples and conducting further research helped me understand how MongoDB stores flexible documents and how queries retrieve event information.

Portfolio 5: Distributed and Cloud Database Exercise

I proposed a MongoDB replica set with three voting members across separate hosts or failure zones: one primary and two secondaries. The design considered elections, majority voting and `w: "majority"` write concern, alongside replication, sharding, cloud deployment, security, backups and cost.

This was a proposed design, not a deployment. I did not run a three-member replica set or perform failover, replication or cloud performance tests.

Evidence

Portfolio evidence in the GitHub repository includes the database design and SQL files; query, optimisation and execution-plan screenshots; transaction and concurrency scripts (`SessionA.sql` and `SessionB.sql`); MongoDB setup, event scripts and screenshots; and the proposed distributed-database design.

All evidence is available in the GitHub repository.

Individual reflection

Completing these portfolios improved my understanding of database design and implementation. One important design choice was using PostgreSQL for core banking records and MongoDB for transaction event documents. This helped me appreciate that different database technologies can serve different purposes within the same platform.

I found the events and queries exercise in Portfolio 4 challenging because I initially needed a better understanding of how MongoDB documents were structured and queried. I addressed this by reviewing the examples and conducting further research.

One of my most important lessons was that the efficiency of a good database starts with its design. A well-designed database makes it easier to manage data, retrieve information and maintain consistency.

References

PostgreSQL: Constraints — https://www.postgresql.org/docs/16/ddl-constraints.html
PostgreSQL: Using EXPLAIN — https://www.postgresql.org/docs/current/using-explain.html
PostgreSQL: Transaction Isolation — https://www.postgresql.org/docs/current/transaction-iso.html
PostgreSQL: Explicit Locking — https://www.postgresql.org/docs/current/explicit-locking.html
MongoDB: Insert Multiple Documents — https://www.mongodb.com/docs/manual/reference/method/db.collection.insertMany/
MongoDB: Query Documents — https://www.mongodb.com/docs/manual/tutorial/query-documents/
MongoDB: Replication — https://www.mongodb.com/docs/manual/replication/
MongoDB: Replica Set Elections — https://www.mongodb.com/docs/v8.0/core/replica-set-elections/
MongoDB: Write Concern — https://www.mongodb.com/docs/v8.0/reference/write-concern/

AI-use declaration

I used ChatGPT as a learning and support tool during the completion of the portfolios. It helped explain database concepts, generate fictional data for the exercises and troubleshoot errors encountered during implementation.

I personally ran and reviewed the SQL and MongoDB commands and checked their outputs before using them as evidence. I used the explanations and suggestions to support my understanding and complete the practical exercises. The final work and evidence reflect the commands I executed and the results I reviewed.